

練習 15：Function Module——建立、測試、呼叫

授課順序：接在練習 8 (FORM) 之後。講義見 [lec15](#)。

學習目標

- 理解 FM 的定位：跨程式共用的邏輯單位 (FORM 只能在同一支程式內用)
- 會在 SE37 建立 Function Group 與 Function Module
- 會設計 FM 介面：IMPORTING / EXPORTING / EXCEPTIONS
- 會在 SE37 單獨測試 FM (不用寫呼叫程式就能測——這是 FM 最大的優點之一)
- 會 CALL FUNCTION 呼叫並處理例外 (EXCEPTIONS ... = 1 與 sy-subrc)
- 搞清楚方向反轉：FM 的 IMPORTING (它收的) = 呼叫端的 EXPORTING (我送的)
- 會用 CHANGING + DDIC Table Type 傳遞一整張表——理解為什麼這是取代舊式 TABLES 參數的現代寫法
- 理解一個 Function Group 可以放多個 FM，並會用 TOP include 的全域變數讓同 group 的 FM 共用、傳遞資料

事前準備

1. SE80 建 Function Group ZFG_TR15_<縮寫> (套件 \$TMP) ——FM 一定要掛在某個 Function Group 下
2. 依團隊命名慣例：Function Group ZFG_xxx、FM Z_xxx

題目需求

Part 1：建 FM (SE37)

建立 Z_TR15_<縮寫>_CALC_REVENUE：

介面	參數	型別
IMPORTING	IV_PRICE	S_PRICE (票價)
IMPORTING	IV_SEATSOCC	S_SEATSOCC (已售座位)
EXPORTING	EV_REVENUE	S_PRICE (營收)
EXCEPTIONS	INVALID_INPUT	票價或座位為負數時 RAISE

邏輯：驗證輸入 → $ev_revenue = iv_price * iv_seatsocc$ 。

Part 2：SE37 單測

SE37 → Test/Execute (F8) → 輸入 1500 / 200 → 確認 EV_REVENUE = 300000；再輸入 -99 → 確認丟出 INVALID_INPUT。

Part 3：寫呼叫程式

1. 正常呼叫：印出營收 (記得方向：值送進 FM 用 EXPORTING、接結果用 IMPORTING)
2. 故意傳負數：EXCEPTIONS invalid_input = 1 OTHERS = 2，檢查 sy-subrc 並輸出訊息

Part 4：CHANGING + Table Type (取代舊式 TABLES 參數)

TABLES 參數是官方標記的過時語法 (原因見講義 15 第 3.1 節) , 新 FM 該用 **CHANGING** 搭配一個正常的 Table Type :

SE11 建 Structure / Table Type 的步驟見講義 25 第 6.6 節 ; 新舊寫法對照見講義 15 第 3.2 節。

1. 建一個 DDIC 結構 (如 **Z_TR15_<縮寫>_FLIGHT_REV**) : **CARRID** / **CONNID** / **PRICE** / **SEATSOCC** / **REVENUE** / **CURRENCY** (**PRICE** / **REVENUE** 是金額欄位 , 記得帶 **CURRENCY** 當參考幣別欄位 , 不然啟用會報「specify reference table AND reference field」)
2. 建一個 DDIC Table Type (如 **Z_TR15_<縮寫>_TT_FLIGHT_REV**) , Line Type 指向上面那個結構
3. 建第二支 FM **Z_TR15_<縮寫>_CALC_REVENUE_TAB** : **CHANGING CT_FLIGHTS TYPE <剛建的 Table Type>** , 內部 **LOOP AT ct_flights ASSIGNING <fs>** 逐列把 **REVENUE = PRICE * SEATSOCC** 算出來、直接寫回原表 (不用另外接 **EXPORTING**)
4. 呼叫端 : 組一個至少 3 筆航班的表 (**REVENUE** 先留空) , 呼叫這支 FM , 確認呼叫後表格的 **REVENUE** 欄位都被填上了

Part 5 : 同一個 Function Group 多個 FM , 共用全域變數

1. 在 Function Group (如 **ZFG_TR15_<縮寫>**) 的 **TOP include** (**LZFG_TR15_<縮寫>TOP**) 加全域變數 : **gv_total_revenue TYPE s_price** 、 **gv_call_count TYPE i** (SE80 展開 Function Group → Includes → 雙擊 TOP include)
2. 在同一個 **group** 再建三支 FM :
 - **Z_TR15_<縮寫>_ADD_REVENUE** : **IMPORTING iv_revenue TYPE s_price** , 把它累加進 **gv_total_revenue** 、 **gv_call_count** 加 1
 - **Z_TR15_<縮寫>_GET_TOTAL** : **EXPORTING ev_total TYPE s_price** 、 **ev_count TYPE i** , 只把兩個全域變數讀出來 (這支不做任何累加)
 - **Z_TR15_<縮寫>_RESET_TOTAL** : 無參數 , **CLEAR** 兩個全域變數
3. 呼叫端 : 先 **GET_TOTAL** (印出初始值) → **ADD_REVENUE** 三次 (50000 / 40000 / 36000) → 再 **GET_TOTAL** 印出累計 → **RESET_TOTAL** → 再 **GET_TOTAL**
4. 觀察 : 三次 **ADD_REVENUE** 都沒有把結果傳回呼叫端 , **GET_TOTAL** 卻讀得到——因為同 group 共用同一份全域變數

Part 5 預期輸出 :

```
=== 1) 尚未累加就先讀 ===
累計營收      0.00   / 累加次數  0
=== 2) 累加三筆後 , 由另一支 FM 讀出 ===
累計營收 126,000.00   / 累加次數  3
=== 3) RESET 之後 ===
累計營收      0.00   / 累加次數  0
```

預期輸出 (呼叫程式)

```
票價 1500.00 × 200 座 = 營收      300,000.00
負數票價被 FM 擋下 , sy-subrc =      1
=== CHANGING + Table Type : 逐航班營收 ===
LH 0400   500.00   100 =>  50,000.00 EUR
```

LH	2402	800.00	50 =>	40,000.00	EUR
AA	0017	1,200.00	30 =>	36,000.00	USD

FM vs FORM vs Method (定位速查)

	FORM	Function Module	Method (OOP 課程)
共用範圍	同一支程式	全系統	全系統
單獨測試	不行	SE37 直接測	ABAP Unit
典型用途	程式內部拆邏輯	共用工具、RFC、BAPI	新開發的商業邏輯

選修 (接續 ex08a)：把 Function Group 建在自己的 Package、掛一張新 TR、到 SE10 展開 Task、觀察整組在 TR 裡記錄成哪一筆物件。

思考題

1. `EXCEPTIONS invalid_input = 1` 的 `1` 是什麼意思？呼叫端不寫 `EXCEPTIONS` 區塊、FM 又 `RAISE` 了會發生什麼事？（會 dump——動手試一次，認識 `RAISE_EXCEPTION` 這個 dump）
2. 在 SE37 打開一支 SAP 標準 FM（例如 `CONVERSION_EXIT_ALPHA_INPUT`，數字補前導零），只看介面頁籤：哪些是 `IMPORTING`、哪些是 `EXPORTING`？不看原始碼，你能寫出呼叫它的 `CALL FUNCTION` 嗎？
3. FM 的 `IMPORTING` 參數為什麼慣用 `VALUE(...)`（傳值）？跟 FORM `USING` 預設傳參考的差異是什麼？
4. Part 4 的 `CT_FLIGHTS` 型別為什麼不能直接用函式群組 Top Include 裡宣告的本地 `TYPES`，一定要走 DDIC Table Type？（提示：FM 介面型別要能被呼叫端獨立語法檢查，不依賴這支 FM 所屬 Function Group 有沒有被載入）
- 5.（Part 5）把 `GET_TOTAL` 建到另一個 Function Group，還讀得到 `ADD_REVENUE` 累加的值嗎？為什麼？（提示：全域變數只在同一個 group 內共用）
- 6.（Part 5）呼叫端在同一支程式裡先累加三筆、沒有 `RESET` 就再累加同樣三筆，`GET_TOTAL` 會回傳多少？這說明了 Function Group 全域變數的什麼特性？

答案

見 `z_tr15_calc_revenue.func.abap` / `z_tr15_calc_revenue_tab.func.abap`（兩支 FM 原始碼，SAP 端 `Z_TR15_CALC_REVENUE` / `Z_TR15_CALC_REVENUE_TAB`，皆掛在 Function Group `ZFG_TR15`）與 `zr_tr15_call_fm.prog.abap`（呼叫端，SAP 端 `ZR_TR15_CALL_FM`）。Part 5（同 group 多個 FM 共用全域變數）見 `z_tr15_add_revenue.func.abap` / `z_tr15_get_total.func.abap` / `z_tr15_reset_total.func.abap`、TOP include 快照 `lzfg_tr15top.prog.abap`，以及呼叫端 `zr_tr15_fg_shared.prog.abap`（SAP 端 `ZR_TR15_FG_SHARED`）。Part 4 用到的 DDIC 結構 `ZTR15_FLIGHT_REV` / Table Type `ZTR15_TT_FLIGHT_REV` 快照見 `ztr15_flight_rev.stru.abap`。